

Code-Layout, Readability and Reusability

Date	4 July 2026
Team ID	XXXXXX
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing and indentation used throughout the code	Yes	PEP 8 coding style followed
2	Proper File Structure	Files and folders are logically organized	Yes	Dataset, source code, models, notebooks, Flask app, and deployment files are separated
3	Meaningful Variable Names	Variables clearly represent their purpose	Yes	Descriptive variable names used throughout the project
4	Function / Method Names	Functions are descriptively named	Yes	Functions such as preprocessing(), predict(), and evaluate_model() clearly describe their functionality
5	Code Comments	Comments explain important logic	Yes	Comments added for preprocessing, model training, and prediction steps
6	Modular Design	Code divided into reusable modules	Yes	Separate modules created for preprocessing, training, prediction, evaluation, and utilities
7	No Redundant Code	Duplicate or unnecessary code removed	Yes	Common functionality organized into reusable modules
8	Error Handling	Exceptions handled appropriately	Yes	Basic input validation and exception handling implemented to ensure stable

				application execution.
--	--	--	--	------------------------

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Data Preprocessing Module (preprocessing.py)	Python	Data cleaning and preprocessing	High
2	Model Training Modules (train_logistic.py, train_decision_tree.py, train_random_forest.py, train_xgboost.py)	Python (Scikit-learn & XGBoost)	Training multiple machine learning models	High
3	Prediction Module (predict.py)	Python	Flask application and prediction workflow	High
4	Utility Module (utils.py)	Python	Shared helper functions	Medium
5	Flask Web Application (app.py)	Flask (Python)	User interface and prediction service	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Well-organized project structure with separate folders for datasets, source code, models, notebooks, Flask application, and deployment files.
Readability	5	Clear naming conventions, consistent formatting, and easy-to-understand code.
Reusability	5	Modular design enables reuse of preprocessing, training, prediction, and utility modules.
Documentation / Comments	5	Code is organized with meaningful section comments and supported by comprehensive project documentation.
Overall Score	5 / 5	The project demonstrates a clean, modular, maintainable, and reusable implementation following good coding practices.